# MAT2540, Classwork20, Spring2026

## *11.2 Applications of Trees*

1. *Introduction.* **Binary Search Trees.**

Our primary goal is to implement a searching algorithm that finds items in a list efficiently when the items are totally ordered. This can be accomplished through the use of a binary search tree:

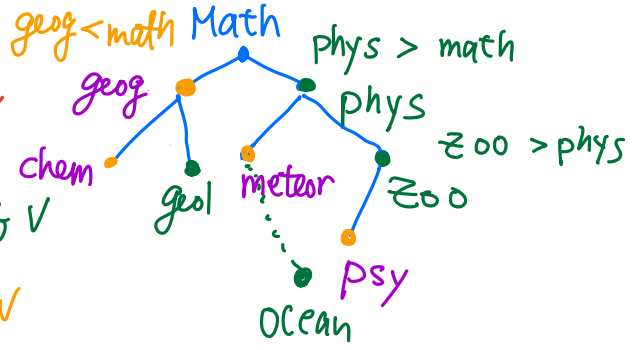
(1) Each child of a vertex is designated as a right child or left child

(2) No vertex has more than one left child or right child.

(3) Vertices are assigned labels so that a vertex's label is both larger than the labels of all vertices in its left subtree and smaller than the labels of all vertices in its right subtree.

2. Form a binary search tree for the words ***math****ematics*, ***phys****ics*, ***geog****raphy*, ***zoo****logy*, ***meteor****ology*, ***geol****ogy*, ***psy****chology*, and ***chem****istry* (using alphabetical order).

**Binary Search Tree**:




**Instructions**:

(1) Start a tree with the root which is the first item in the list

(2) To add a new item $x$, first compare it with the labels of vertices $v$ already in the tree, starting at the root:

(a) $x >$ label($v$): $x$ is the label of the right child of $v$

(b) $x <$ label($v$): $x$ is the label of the left child of $v$

(3) Do this procedure recursively until all items are included

**Pseudocode**: Locating an Item in or Adding an Item to a Binary Search Tree.

```
0  Procedure: insertion(T: binary search tree, x: item)
1  v := root of T {a vertex does not present in T has the value null}
2  while v ≠ null and label(v) ≠ x
3      if x < label(v) then
4          if left child of v ≠ null then v := the left child of v
5          else add new vertex as a left child of v and set v := null
6      else
7          if right child of v ≠ null then v := the right child of v
8          else add new vertex as a right child of v and set v := null
9  if root of T = null then add a vertex v to the tree and label it with x
10 else if v is null and label(v) ≠ x then label new vertex with x and let v be this new vertex
11 return v{ v = location of x}
```

3. Use the Algorithm in 2. to insert the word ***ocean****ography* into the binary search tree in the previous example.

Step 1 v = root (line 1)
(line 2) v ≠ null, label(v) = Math ≠ ocean
(line 6) ocean > math
(line 7) v = right child of root, label(v) = phys

step 2 (line 2) v ≠ null, label(v) = phys ≠ ocean
(line 3) ocean < phys
(line 4) v = left child of phys, label(v) = meteor

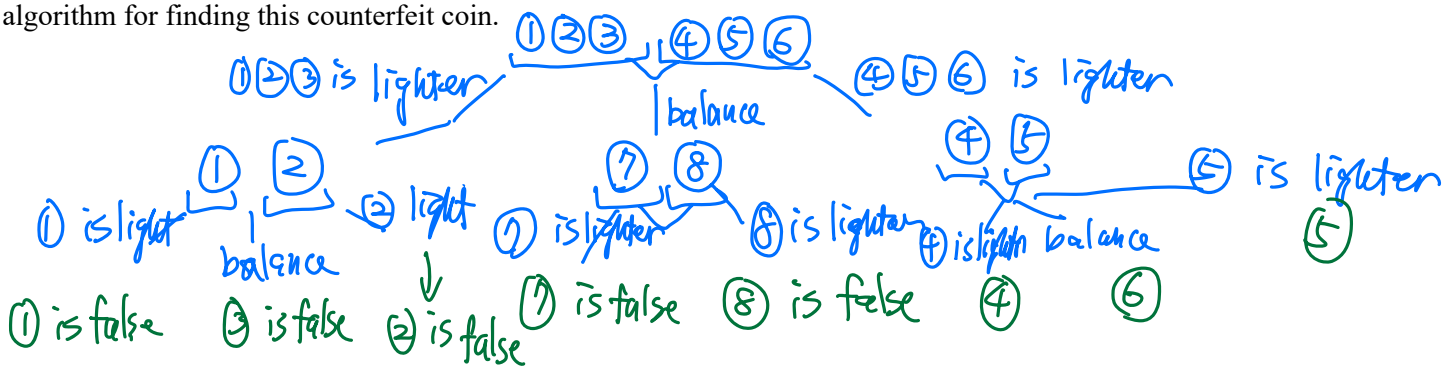
step 3 (line 2) v ≠ null, label(v) = meteor ≠ ocean
(line 6) ocean > meteor
(line 8) create a v as the right child of meteor
v = null

step 4 (line 2) since v = null. While STOP
(line 10) label(v) = ocean
(line 11) return

4. *Introduction.* **Decision Trees**.

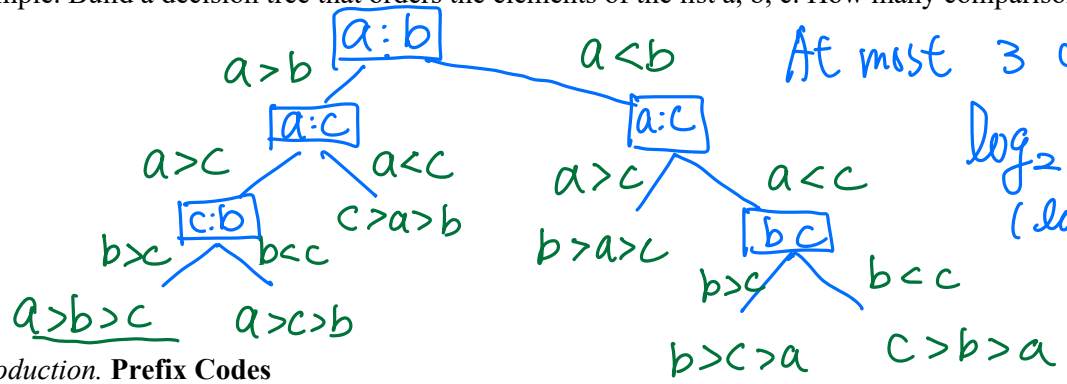
A rooted tree in which each internal vertex corresponds to a decision, with a subtree at these vertices for each possible outcome of the decision, is called a decision tree.

5. Suppose there are seven coins, all with the same weight, and a counterfeit coin that weighs less than the others. How many weighings are necessary using a balance scale to determine which of the eight coins is the counterfeit one? Give an algorithm for finding this counterfeit coin.




6. *Theorem*. To sort a list with $n$ elements, a sorting algorithm based on binary comparisons requires at least $\lceil \log_2 n! \rceil$ comparisons.

7. Example. Build a decision tree that orders the elements of the list a, b, c. How many comparisons are needed?




At most 3 comparisons

$\log_2 3! = \log_2 6 = 2.\cdots$

$(\log_2 4 = 2, \log_2 8 = 3)$

$\lceil \log_2 3! \rceil = \lceil 2.\cdots \rceil = 3$

8. *Introduction.* **Prefix Codes**

Consider the problem of using bit strings to encode the letters of the English alphabet.

(1) Ensure that no bit string corresponds to more than one sequence of letters

(2) Letters that occur more frequently should be encoded using short bit strings, and longer bit strings should be used to encode rarely occuring letters.

9. Example.

(1) Consider this code: $e$: 0, $a$: 1, $t$: 01. What does the bit string 0101 correspond to?

0101 — e a t

0101 — t e a

0101 — t t

0101 — e a e a

(2) Consider this code: $e$: 0, $a$: 10, $t$: 11. What does the bit string 10110 correspond to?

10110 — a t e

10. How to construct a prefix code?

We can construct a prefix code from any binary tree where the left edge labels as 0, the right edge labels as 1, and only put the letter on leaf vertex. Characters are encoded with the bit string constructed using the labels of edges in the unique path from the root to the leaves.